

DOM Manipulation Notes

What is DOM?

DOM stands for Document Object Model

- The Document Object Model (DOM) is a programming interface for web documents that represents the HTML or XML document as a tree structure, where each node represents an element, attribute, or piece of text in the document.
- When a web page is loaded, the browser creates a DOM tree that represents the document's structure and content.
- Each node in the tree is represented as js object, which we can access and manipulate using the DOM API.
- Here, Html elements, comments, text, content, etc are referred as nodes of DOM tree.
- The Document Object Model (DOM) is the data representation of the objects that comprise the structure and content of a document on the web
- An HTML/XML document is represented inside the browser as the DOM tree. Tags become element nodes and form the structure. Text becomes text nodes
- Think of DOM as a bridge between your HTML code and JavaScript
- When browser loads your webpage, it creates a tree-like structure called DOM tree

Why JavaScript Alone is Slow?

- JavaScript by itself is slow because it re-renders the page continuously
- Even small changes cause the entire page to refresh
- React is faster than plain JavaScript because it uses Virtual DOM
- React updates only the parts that actually changed, not the whole page

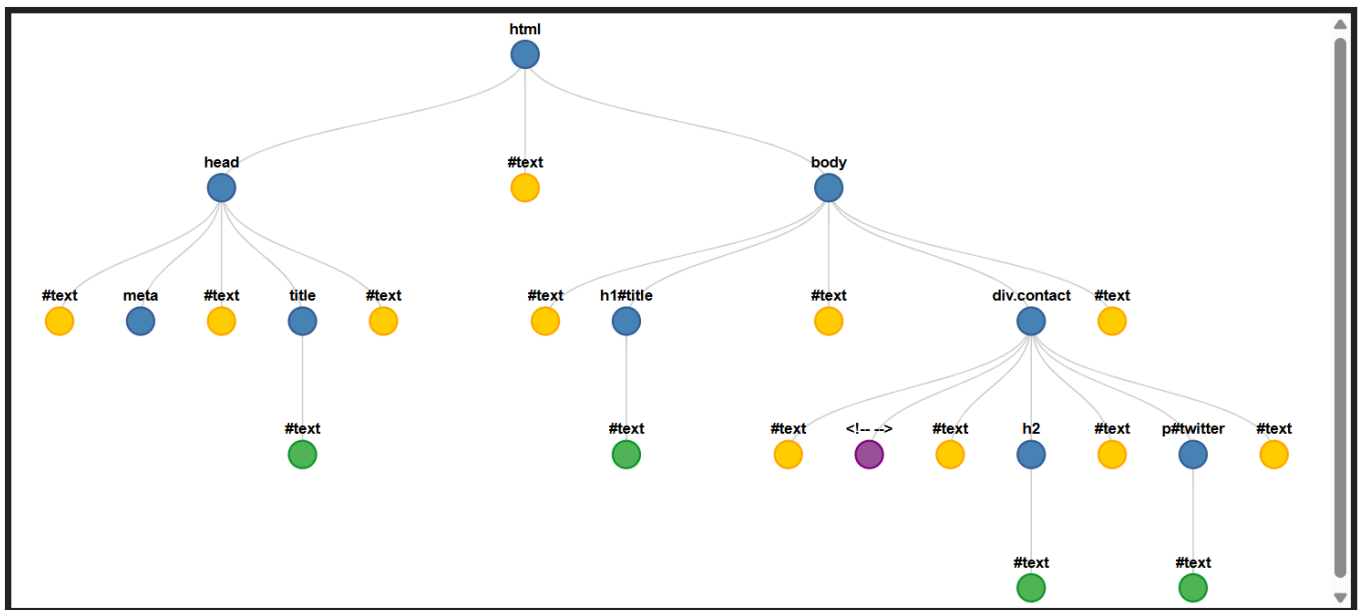
How DOM Tree Works?

The HTML Document Object Model (DOM) is a tree structure, where each HTML tag becomes a node in the hierarchy

Example:

```
<!DOCTYPE html>
<html>           ← Root Node
  <head>         ← Child of html
    <title>My Page</title> ← Child of head
  </head>
  <body>         ← Child of html, Sibling of head
    <h1>Hello</h1>   ← Child of body
    <p>World!</p>    ← Child of body, Sibling of h1
  </body>
</html>
```

Important: Even empty spaces (like when you press Enter) are considered as text nodes in DOM!



```

1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta charset="UTF-8">
5     <title>DOM Visualizer</title>
6   </head>
7   <body>
8     <h1 id="title">DOM Visualizer</h1>
9     <div class="contact">
10      <!-- This is a comment -->
11      <h2>Romain Bohdanowicz</h2>
12      <p id="twitter">Twitter : @bioub</p>
13    </div>
14  </body>
15 </html>

```

Types of Nodes in DOM

The DOM is structured as a tree of nodes that will usually be HTML elements, text, or comments

1. **Element Node** - HTML tags like `<h1>`, `<p>`, `<div>`
2. **Text Node** - The actual text content inside elements
3. **Attribute Node** - Properties like `id`, `class`, `src`
4. **Comment Node** - HTML comments `<!-- like this -->`
5. **Empty Space Node** - Whitespace and line breaks

Key Point: Each node is a JavaScript object with properties and methods!

DOM API - The Bridge

What is DOM API?

- DOM API is the intermediate layer between JavaScript and the browser
- It's a set of programming interfaces that allow developers to communicate with the DOM tree
- DOM methods allow programmatic access to the tree. With them, you can change the document's structure, style, or content

How it works:

1. Your JavaScript code goes to V8 engine
2. But to change webpage, it needs to go to rendering engine
3. DOM API (part of Web APIs) makes this communication possible
4. DOM tree gets updated only through DOM API functions

Browser Requirements

Any browser in the world only needs three types of files:

- `.html` file (structure)
- `.css` file (styling)
- `.js` file (functionality)

What are Bundlers?

Bundler is a tool that:

- Combines multiple files of different types
- Gives the browser only the 3 files it needs (.html, .css, .js)
- In simple terms: bundler merges multiple files into optimized bundles

Popular Bundlers:

- **Webpack** - Most popular, highly configurable
- **Parcel** - Easy to use, zero configuration
- **Vite** - Very fast, modern
- **Rollup** - Fast and efficient

DOM Manipulation Methods

1. Selecting Elements

```
// By ID
const element = document.getElementById("myId");

// By Class Name
const elements = document.getElementsByClassName("myClass");

// By Tag Name
const elements = document.getElementsByTagName("p");

// Modern Selectors (Recommended)
const element = document.querySelector("#myId");
// It returns reference of the first element that matches a specified CSS selector.

const elements = document.querySelectorAll(".myClass");
// It returns Nodelist of all elements that matches a specified CSS selector.
```

2. Changing Content

```
// Change text content
element.textContent = "New text content";

// Change HTML content
element.innerHTML = "<strong>Bold text</strong>";

// Get current content
console.log(element.textContent); // Gets text only
console.log(element.innerHTML);   // Gets HTML code
```

3. Styling Elements

There are two main ways of styling elements when working with the DOM in JavaScript. You can use the `.style` property or you can use classes

Method 1: Direct Styling

```
element.style.color = "red";
element.style.fontSize = "40px";
element.style.backgroundColor = "blue";
```

Method 2: CSS Classes (Recommended)

```
// Add class
element.classList.add("highlight");

// Remove class
element.classList.remove("highlight");

// Toggle class
element.classList.toggle("active");

// Check if class exists
if(element.classList.contains("highlight")) {
  console.log("Element has highlight class");
}
```

4. Creating New Elements

```
// Create new element
const newDiv = document.createElement("div");

// Add content
newDiv.textContent = "I am a new div!";

// Add attributes
newDiv.id = "newElement";
newDiv.className = "highlight";

// Add to page
document.body.appendChild(newDiv);
```

5. Removing Elements

```
// Remove element
element.remove();

// Or remove child from parent
parent.removeChild(childElement);
```

6. Changing Attributes

```
// Set attribute
element.setAttribute("src", "image.jpg");
element.setAttribute("alt", "My Image");

// Get attribute
const srcValue = element.getAttribute("src");

// Remove attribute
element.removeAttribute("alt");

// For common attributes, use direct properties
element.id = "myId";
element.src = "image.jpg";
element.href = "https://example.com";
```

Practical Example

HTML File (index.html):

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DOM Manipulation</title>
</head>
<body>
  <h1 id="title">Namaste Developers</h1>
  <p class="description">Learning DOM manipulation</p>
  <button id="changeBtn">Change Content</button>

  <script src="script.js"></script>
</body>
</html>
```

JavaScript File (script.js):

```
// Get elements
const h1 = document.getElementById("title");
const description = document.querySelector(".description");
const button = document.getElementById("changeBtn");

// Change h1 styling and content
h1.style.color = "red";
h1.style.fontSize = "40px";
h1.textContent = "Hello Developers";

// Add click event to button
button.addEventListener("click", function() {
  // Toggle between two messages
  if (h1.textContent === "Hello Developers") {
    h1.textContent = "Welcome to DOM!";
    h1.style.color = "blue";
  } else {
    h1.textContent = "Hello Developers";
    h1.style.color = "red";
  }
});

// Create and add new element
const newParagraph = document.createElement("p");
newParagraph.textContent = "This paragraph was created with JavaScript!";
newParagraph.style.fontWeight = "bold";
document.body.appendChild(newParagraph);
```

Event Handling

```
// Click event
element.addEventListener("click", function() {
  console.log("Element clicked!");
});

// Multiple events
element.addEventListener("mouseover", function() {
  element.style.backgroundColor = "yellow";
});

element.addEventListener("mouseout", function() {
  element.style.backgroundColor = "white";
});

// Form events
const form = document.querySelector("form");
form.addEventListener("submit", function(event) {
  event.preventDefault(); // Prevent form submission
  console.log("Form submitted!");
});
```

DOM Navigation

With the HTML DOM, all nodes in the node tree can be accessed by JavaScript. New nodes can be created, and all nodes can be modified or deleted

```
// Parent-Child Relationships
const parent = element.parentNode;
const children = element.childNodes;
const firstChild = element.firstChild;
const lastChild = element.lastChild;

// Sibling Relationships
const nextSibling = element.nextSibling;
const previousSibling = element.previousSibling;

// More specific (ignoring text nodes)
const parentElement = element.parentElement;
const childElements = element.children;
const firstElementChild = element.firstElementChild;
const nextElementSibling = element.nextElementSibling;
```

Best Practices

1. **Use Modern Selectors:** Prefer `querySelector()` and `querySelectorAll()`
2. **Cache DOM Elements:** Store frequently used elements in variables
3. **Use CSS Classes:** Instead of direct styling when possible
4. **Event Delegation:** For dynamic content, use event delegation
5. **Minimize DOM Manipulations:** Batch changes together for better performance

Performance Tips

1. **Cache Elements:**

```
// Bad - searches DOM every time
document.getElementById("myButton").style.color = "red";
document.getElementById("myButton").textContent = "Click me";

// Good - cache the element
const button = document.getElementById("myButton");
button.style.color = "red";
button.textContent = "Click me";
```

2. **Use Document Fragments for Multiple Additions:**

```
const fragment = document.createDocumentFragment();
for (let i = 0; i < 100; i++) {
  const div = document.createElement("div");
  div.textContent = `Item ${i}`;
  fragment.appendChild(div);
}
document.body.appendChild(fragment); // Add all at once
```

Common Mistakes to Avoid

1. Forgetting to check if element exists:

```
// Bad
const element = document.getElementById("nonexistent");
element.style.color = "red"; // Error!

// Good
const element = document.getElementById("myId");
if (element) {
  element.style.color = "red";
}
```

2. Modifying DOM inside loops unnecessarily

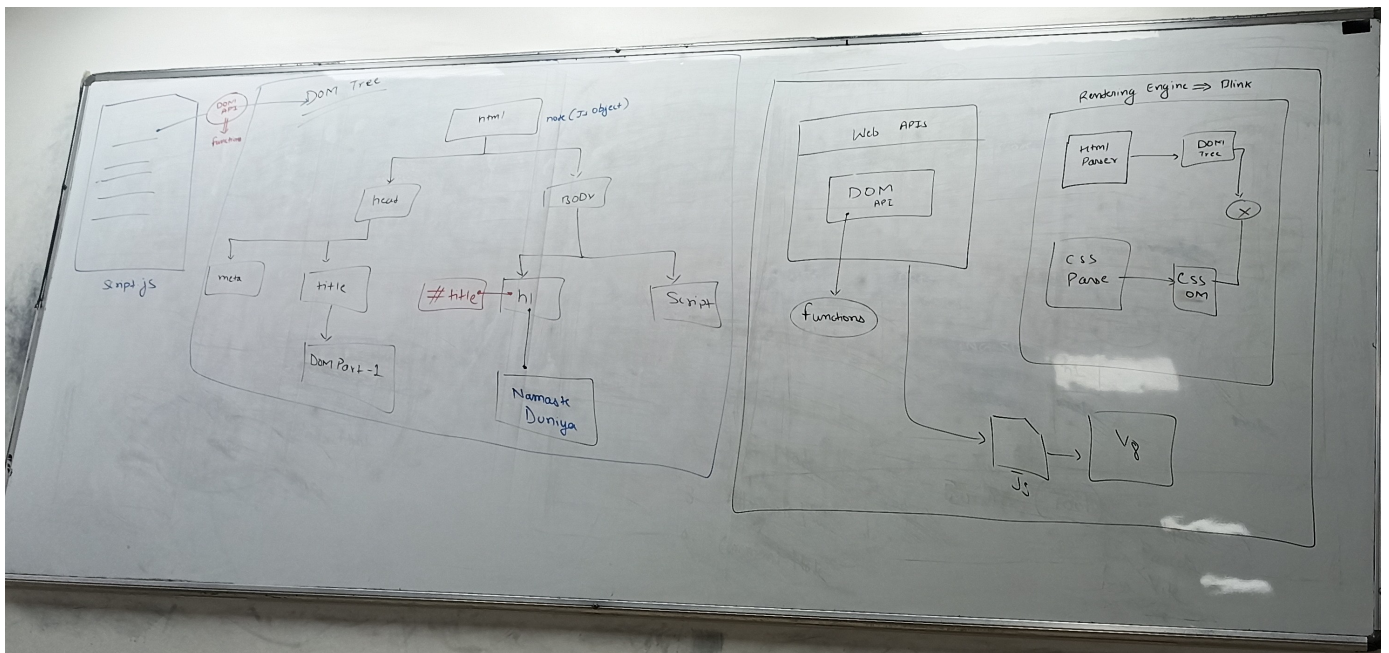
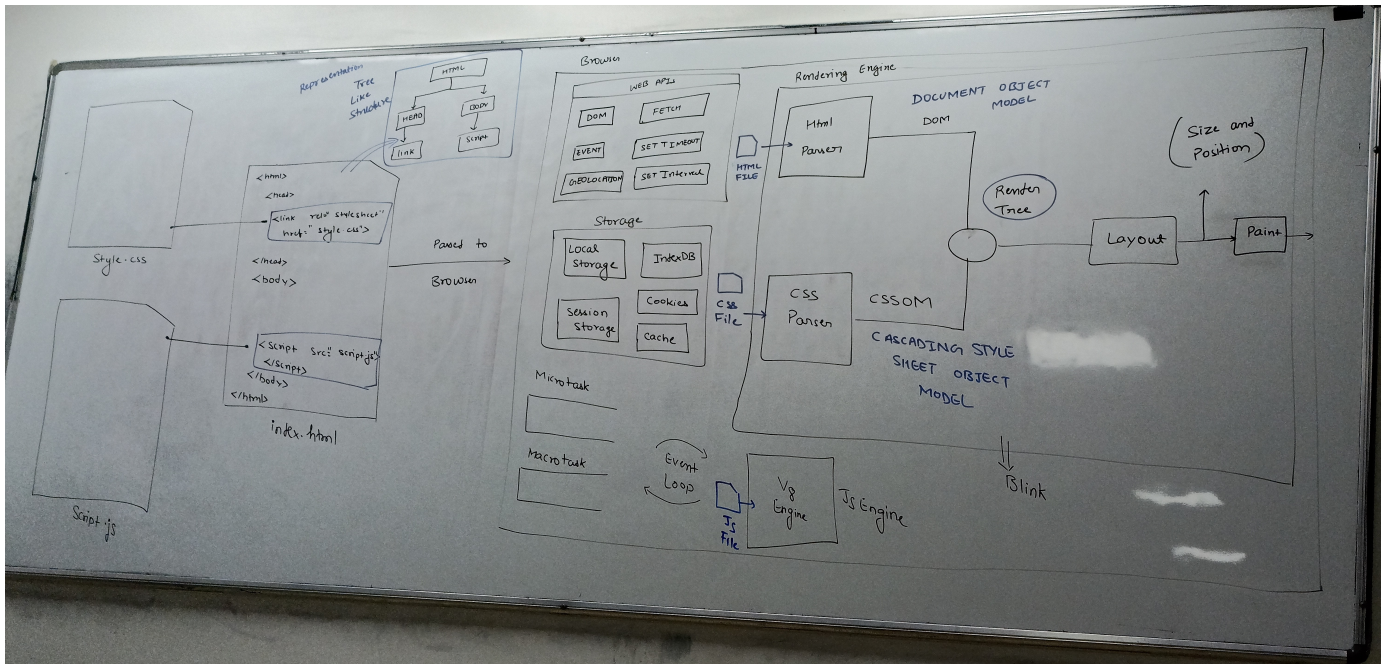
3. Not using event delegation for dynamic content

4. Mixing JavaScript with HTML (inline event handlers)

Summary

- DOM is the bridge between HTML and JavaScript
- Everything in HTML becomes a node (object) in DOM tree
- DOM API provides methods to manipulate these nodes
- Modern browsers need only HTML, CSS, and JS files
- Use bundlers to optimize and combine multiple files
- Always cache DOM elements for better performance
- Prefer CSS classes over direct styling for maintainable code

Remember: DOM manipulation allows developers to interact with web pages and create dynamic and interactive experiences. By understanding how to select, create, modify, and delete elements, you gain full control over the page's appearance and behavior



Complete DOM Manipulation Guide

Table of Contents

1. Element Selection Methods
 2. HTML Collections vs NodeLists
 3. Creating and Modifying Elements
 4. Interview Questions & Answers
-

Element Selection Methods

1. `document.getElementById()`

Purpose: Selects a single element by its unique ID attribute.

Syntax: `document.getElementById("elementId")`

Returns: Single element or `null` if not found

```
// HTML: <h1 id="title">Hello World</h1>

const heading = document.getElementById("title");
heading.textContent = "Namaste Developers";
heading.style.color = "red";

console.log("heading:", heading);
// Output: <h1 id="title">Namaste Developers</h1>
```

Key Points:

- Returns only the **first** element with that ID
 - IDs should be unique in HTML
 - Most efficient selection method
 - Returns `null` if element doesn't exist
-

2. `document.getElementsByClassName()`

Purpose: Selects all elements with a specific class name.

Syntax: `document.getElementsByClassName("className")`

Returns: Live HTMLCollection (array-like object)

```
// HTML:
// <div class="card">Card 1</div>
// <div class="card">Card 2</div>
// <div class="card">Card 3</div>

// Method 1: Direct HTMLCollection (not a true array)
const cards = document.getElementsByClassName("card");
console.log(cards); // HTMLCollection(3)

// Method 2: Convert to Array for array methods
const arr = Array.from(document.getElementsByClassName("card"));
console.log("arr:", arr); // [div.card, div.card, div.card]

// Now we can use array methods
arr.map((element, index) => {
  element.style.color = "red";
  element.textContent = `Modified Card ${index + 1}`;
});

// Alternative: Using forEach with HTMLCollection
Array.from(cards).forEach((element, index) => {
  element.style.backgroundColor = "lightblue";
});
```

Key Points:

- Returns **live** HTMLCollection
- Automatically updates when DOM changes
- Need `Array.from()` to use array methods
- Case-sensitive class names

3. `document.getElementsByTagName()`

Purpose: Selects all elements with a specific tag name.

Syntax: `document.getElementsByTagName("tagName")`

Returns: Live HTMLCollection

```
// HTML: Multiple <div> elements

const divs = document.getElementsByTagName("div");
console.log(divs); // HTMLCollection of all div elements

// Convert to array to modify all divs
const divArray = Array.from(divs);
divArray.forEach((div, index) => {
  div.style.color = "blue";
  div.style.border = "2px solid blue";
});

// Select specific elements
const paragraphs = document.getElementsByTagName("p");
const images = document.getElementsByTagName("img");
```

Key Points:

- Selects ALL elements with that tag
 - Case-insensitive for HTML documents
 - Returns live HTMLCollection
 - Very broad selection method
-

4. `document.querySelector()`

Purpose: Selects the **first** element matching a CSS selector.

Syntax: `document.querySelector("cssSelector")`

Returns: Single element or `null`

```
// Select by ID
const heading = document.querySelector("#title");
console.log(heading);

// Select by class
const firstCard = document.querySelector(".card");

// Select by tag
const firstDiv = document.querySelector("div");

// Complex selectors
const specificCard = document.querySelector(".container > .card:nth-child(3)");
const nestedElement = document.querySelector(".container .card p");

// Attribute selectors
const inputEmail = document.querySelector("input[type='email']");
const linkExternal = document.querySelector("a[href^='http']");
```

Key Points:

- Uses CSS selector syntax
 - Returns only the **first** matching element
 - Very flexible and powerful
 - Returns `null` if no match found
-

5. `document.querySelectorAll()`

Purpose: Selects **all** elements matching a CSS selector.

Syntax: `document.querySelectorAll("cssSelector")`

Returns: Static NodeList

```
// Select all cards
const cards = document.querySelectorAll(".card");
console.log(cards); // NodeList(3)

// NodeList has forEach method built-in
cards.forEach((element, index) => {
  element.style.color = "blue";
  element.textContent = `Card ${index + 1}`;
});

// Complex selections
const allLinks = document.querySelectorAll("a");
const externalLinks = document.querySelectorAll("a[href^='http']");
const evenCards = document.querySelectorAll(".card:nth-child(even)");

// Convert to array if needed
const cardsArray = Array.from(cards);
const cardsArraySpread = [...cards];
```

Key Points:

- Returns **static** NodeList
- Has built-in `forEach()` method
- Snapshot of elements at query time
- Most versatile selection method

HTML Collections vs NodeLists

HTMLCollection

```
const cardHtmlCollection = document.getElementsByClassName("card");

// Live collection example
const c4 = document.getElementById("c4");
console.log("Before removal:", cardHtmlCollection.length); // 4

c4.remove();
console.log("After removal:", cardHtmlCollection.length); // 3 (automatically updated)
```

Characteristics:

- **Live:** Automatically updates when DOM changes
- **Array-like:** Has length and numeric indices
- **Limited methods:** No `forEach`, `map`, `filter`
- **Element-only:** Contains only element nodes

Methods available:

- `collection.length`
- `collection.item(index)`
- `collection.namedItem(name)`

NodeList

```
const cardNodeList = document.querySelectorAll(".card");

// Static collection example
const c4 = document.getElementById("c4");
console.log("Before removal:", cardNodeList.length); // 4

c4.remove();
console.log("After removal:", cardNodeList.length); // 4 (unchanged - static snapshot)
```

Characteristics:

- **Static:** Snapshot at query time (for `querySelectorAll`)
- **Array-like:** Has `length` and numeric indices
- **More methods:** Has `forEach` built-in
- **Any node type:** Can contain elements, text nodes, comments

Methods available:

- `nodeList.length`
- `nodeList.item(index)`
- `nodeList.forEach(callback)`
- `nodeList.keys()`
- `nodeList.values()`
- `nodeList.entries()`

Visual Comparison

```
// Live vs Static demonstration
const liveCollection = document.getElementsByClassName("test");
const staticNodeList = document.querySelectorAll(".test");

console.log("Initial - Live:", liveCollection.length); // 3
console.log("Initial - Static:", staticNodeList.length); // 3

// Add new element
const newDiv = document.createElement("div");
newDiv.className = "test";
document.body.appendChild(newDiv);

console.log("After addition - Live:", liveCollection.length); // 4 (updated)
console.log("After addition - Static:", staticNodeList.length); // 3 (unchanged)
```

Creating and Modifying Elements

1. `document.createElement()`

Purpose: Creates a new HTML element.

Syntax: `document.createElement("tagName")`

```
// Create elements
const heading = document.createElement("h1");
const paragraph = document.createElement("p");
const button = document.createElement("button");
const div = document.createElement("div");

// Set properties
heading.textContent = "Namaste Developers";
heading.id = "main-heading";
heading.className = "title-class";

paragraph.innerHTML = "This is a <strong>paragraph</strong>";
button.textContent = "Click Me";
```

2. `appendChild()` vs `append()`

`appendChild()`

- Adds **one** child element
- Returns the appended element
- Only accepts **Node** objects

```
const heading = document.createElement("h1");
heading.textContent = "Namaste Developers";

const body = document.body;
body.appendChild(heading); // Adds to end of body

// Chaining example
const container = document.createElement("div");
const paragraph = document.createElement("p");
paragraph.textContent = "Hello World";

document.body.appendChild(container).appendChild(paragraph);
```

`append()`

- Adds **multiple** children
- Can add **strings** and **nodes**
- No return value (void)

```
const container = document.createElement("div");
const heading = document.createElement("h1");
const paragraph = document.createElement("p");

heading.textContent = "Title";
paragraph.textContent = "Content";

// Append multiple elements and text
container.append(heading, paragraph, "Some text");
document.body.append(container);

// Mixed content
const section = document.createElement("section");
section.append("Section Title: ", heading, paragraph);
```

3. Other Insertion Methods

```
const container = document.getElementById("container");
const newElement = document.createElement("div");
newElement.textContent = "New Element";

// Insert at beginning
container.prepend(newElement);

// Insert before/after specific element
const referenceElement = document.querySelector(".reference");
container.insertBefore(newElement, referenceElement);

// Modern insertion methods
referenceElement.before(newElement); // Insert before
referenceElement.after(newElement);   // Insert after
referenceElement.replaceWith(newElement); // Replace element
```

4. Complete Example: Dynamic Content Creation

```
// Create a complete card component
function createCard(title, content, imageUrl) {
  // Create elements
  const card = document.createElement("div");
  const cardHeader = document.createElement("div");
  const cardTitle = document.createElement("h3");
  const cardImage = document.createElement("img");
  const cardBody = document.createElement("div");
  const cardContent = document.createElement("p");
  const cardButton = document.createElement("button");

  // Set classes
  card.className = "card";
  cardHeader.className = "card-header";
  cardBody.className = "card-body";
  cardButton.className = "btn btn-primary";

  // Set content
  cardTitle.textContent = title;
  cardImage.src = imageUrl;
  cardImage.alt = title;
  cardContent.textContent = content;
  cardButton.textContent = "Learn More";

  // Build structure
  cardHeader.append(cardTitle, cardImage);
  cardBody.append(cardContent, cardButton);
  card.append(cardHeader, cardBody);

  return card;
}

// Use the function
const myCard = createCard(
  "JavaScript DOM",
  "Learn about DOM manipulation in JavaScript",
  "https://via.placeholder.com/300x200"
);

document.body.appendChild(myCard);
```

Interview Questions & Answers

Q1: What's the difference between HTMLCollection and NodeList?

Answer:

HTMLCollection	NodeList
Live - automatically updates	Static - snapshot at query time
Contains only element nodes	Contains any node type (elements, text, comments)
Limited methods (length, item)	More methods (forEach, keys, values)
From <code>getElementsBy*</code> methods	From <code>querySelectorAll</code>

Example:

```
// HTMLCollection (Live)
const liveCards = document.getElementsByClassName("card"); // Live

// NodeList (Static)
const staticCards = document.querySelectorAll(".card"); // Static

// Add new card
const newCard = document.createElement("div");
newCard.className = "card";
document.body.appendChild(newCard);

console.log(liveCards.length); // Increases by 1
console.log(staticCards.length); // Stays the same
```

Q2: When should you use `querySelector` vs `getElementById`?

Answer:

Use `getElementById` when:

- You have a specific ID
- Performance is critical (fastest method)
- Simple selection needed

Use `querySelector` when:

- You need complex CSS selectors
- Selecting by attributes, pseudo-classes
- Consistency in code style

```
// getElementById - fastest for IDs
const header = document.getElementById("main-header");

// querySelector - more flexible
const firstCard = document.querySelector(".card:first-child");
const emailInput = document.querySelector("input[type='email']");
```

Q3: What's the difference between `appendChild` and `append`?

Answer:

<code>appendChild</code>	<code>append</code>
Adds one node only	Adds multiple nodes/strings
Returns the appended node	Returns undefined
Node objects only	Accepts strings and nodes

```
const container = document.createElement("div");

// appendChild - one element, returns element
const heading = document.createElement("h1");
const returnedElement = container.appendChild(heading);
console.log(returnedElement === heading); // true

// append - multiple items, no return value
const paragraph = document.createElement("p");
container.append(paragraph, "Some text", heading); // void return
```

Q4: How do you convert HTMLCollection to Array?

Answer:

```
const htmlCollection = document.getElementsByClassName("item");

// Method 1: Array.from()
const array1 = Array.from(htmlCollection);

// Method 2: Spread operator
const array2 = [...htmlCollection];

// Method 3: Array.prototype.slice.call()
const array3 = Array.prototype.slice.call(htmlCollection);

// Now you can use array methods
array1.forEach(item => console.log(item));
array2.map(item => item.style.color = "red");
```

Q5: What's the performance difference between selection methods?

Answer (from fastest to slowest):

1. `getElementById` - Fastest (hash table lookup)
2. `querySelector` - Fast (optimized CSS engine)
3. `getElementsByClassName` - Medium
4. `getElementsByTagName` - Medium
5. `querySelectorAll` - Slower (full CSS parsing)

Benchmark Example:

```
// Fastest
const fastElement = document.getElementById("specific-id");

// Good performance
const goodElement = document.querySelector("#specific-id");

// Slower for simple selections
const slowElements = document.querySelectorAll("#specific-id");
```

Q6: How do you safely remove elements from a live HTMLCollection?

Answer:

```
// Problem: Removing from live collection while iterating
const items = document.getElementsByClassName("remove-me");

// ✗ Wrong way - skips elements as collection shrinks
for (let i = 0; i < items.length; i++) {
  items[i].remove(); // Collection length changes during loop!
}

// ✓ Correct way 1: Convert to static array first
const itemsArray = Array.from(items);
itemsArray.forEach(item => item.remove());

// ✓ Correct way 2: Remove from end to beginning
for (let i = items.length - 1; i >= 0; i--) {
  items[i].remove();
}

// ✓ Correct way 3: Always remove the first item
while (items.length > 0) {
  items[0].remove();
}
```

Best Practices Summary

1. Use `getElementById` for single elements by ID - fastest performance
2. Use `querySelector/querySelectorAll` for complex selections - most flexible
3. Convert HTMLCollections to arrays when you need array methods
4. Be careful with live collections when modifying DOM during iteration
5. Use `textContent` instead of `innerHTML` when you don't need HTML parsing
6. Cache DOM selections in variables to avoid repeated queries
7. Use `append()` for multiple elements, `appendChild()` for single elements

1. insertAdjacentElement(position, element)

The `insertAdjacentElement()` method inserts a given element node at a specified position relative to the element it is invoked upon.

Syntax

```
element.insertAdjacentElement(position, elementToInsert)
```

Positions

- `beforebegin` - Before the target element itself
- `afterbegin` - Just inside the target element, before its first child
- `beforeend` - Just inside the target element, after its last child
- `afterend` - After the target element itself

Visual Representation

```
<!-- beforebegin -->
<div id="target">
  <!-- afterbegin -->
  <p>existing content</p>
  <!-- beforeend -->
</div>
<!-- afterend -->
```

Example Code

```
const div = document.getElementById("root");

// Create elements
const p1 = document.createElement("p");
p1.textContent = "Paragraph - 1";

const p3 = document.createElement("p");
p3.textContent = "Paragraph - 3";

// Insert elements
div.insertAdjacentElement("afterbegin", p1); // First child
div.insertAdjacentElement("beforeend", p3); // Last child

// Insert elements outside the target
const t1 = document.createElement("h2");
const t2 = document.createElement("h2");

t1.textContent = "Start";
t2.textContent = "End";
t1.style.color = "green";
t2.style.color = "red";

div.insertAdjacentElement("beforebegin", t1); // Before div
div.insertAdjacentElement("afterend", t2);    // After div
```

2. Attribute Methods

setAttribute(attributeName, attributeValue)

Sets the value of an attribute on the specified element.

```
const div = document.querySelector("div");

// Set a single attribute
div.setAttribute("class", "container");

// To preserve existing attributes, use concatenation
const existingClass = div.getAttribute("class");
div.setAttribute("class", `${existingClass} justify-center`);
```

Important Notes:

- If the attribute already exists, `setAttribute()` replaces the previous value
- To preserve existing values, first get the current value and concatenate

getAttribute(attributeName)

Returns the value of a specified attribute on the element.

```
const div = document.querySelector("div");
const classValue = div.getAttribute("class");
console.log(classValue); // Outputs the class attribute value
```

removeAttribute(attributeName)

Removes the attribute with the specified name from the element.

```
const div = document.querySelector("div");
div.removeAttribute("id"); // Removes the id attribute completely
```

Complete Example

```
const div = document.querySelector("div");

// Get current class
const currentClass = div.getAttribute("class");

// Add new class while preserving existing ones
div.setAttribute("class", `${currentClass} justify-center`);

// Remove an attribute
div.removeAttribute("id");

// Check the result
console.log(div.getAttribute("class"));
```

3. Content Manipulation Properties

innerHTML

Gets or sets the HTML markup contained within the element.

```
const div = document.querySelector("#root");
div.innerHTML = "<h1>Hello world</h1>"; // Replaces all content inside div
```

Characteristics:

- Adds content inside the target element
- Replaces existing content
- Can include HTML tags
- Parses HTML strings into DOM elements

outerHTML

Gets or sets the HTML markup of the element including the element itself.

```
const div = document.querySelector("#root");
div.outerHTML = "<section><h1>New Section</h1></section>";
```

Characteristics:

- Replaces the entire element (including the element itself)
- Creates a completely new structure
- The original element is removed from the DOM

innerText

Gets or sets the text content of the element and its descendants.

```
const div = document.querySelector("#root");
div.innerText = "Plain text content"; // Sets only text, no HTML
```

Characteristics:

- Only handles plain text
- Ignores HTML tags
- Respects CSS styling (hidden elements won't be included)

Comparison Example

```
const div = document.querySelector("#root");

// innerHTML - adds HTML inside the element
div.innerHTML = "<p>HTML content</p>";

// outerHTML - replaces the entire element
div.outerHTML = "<section>New element</section>";

// innerText - sets plain text only
div.innerText = "Just plain text";
```

4. DOM Traversal

Parent Element

```
const div = document.querySelector("#root");
console.log(div.parentElement); // Gets the parent element
```

Child Elements

```
const div = document.querySelector("#root");

console.log(div.children);           // HTMLCollection of child elements (3)
console.log(div.childNodes);        // NodeList including text nodes (7)
console.log(div.firstChild);        // First child element
console.log(div.lastElementChild);  // Last child element
```

Key Difference:

- `children` - Only element nodes
- `childNodes` - All nodes including text nodes (whitespace, line breaks)

Sibling Navigation

```
const div = document.querySelector("#root");

// Navigate to siblings
console.log(div.firstElementChild.nextElementSibling); // Second child (p2)
console.log(div.lastElementChild.previousElementSibling); // Second child (p2)
```

Complete Traversal Example

```
const div = document.querySelector("#root");

// Parent
console.log("Parent:", div.parentElement);

// Current element
console.log("Current:", div);

// Children
console.log("Element children:", div.children); // HTMLCollection
console.log("All child nodes:", div.childNodes); // NodeList

// First and last children
console.log("First child:", div.firstElementChild);
console.log("Last child:", div.lastElementChild);

// Sibling navigation
console.log("Second child:", div.firstElementChild.nextElementSibling);
console.log("Second child (from end):", div.lastElementChild.previousElementSibling);
```

5. Element Removal

remove()

Removes the element from the DOM.

```
const div = document.querySelector("#root");
div.children[2].remove(); // Removes the third child element
```

removeChild(childElement)

Removes a specified child element from the parent.

```
const div = document.querySelector("#root");
const p2 = document.querySelector("#root :nth-of-type(2)");
div.removeChild(p2); // Removes the second paragraph
```

Key Differences:

- `remove()` - Called on the element to be removed
- `removeChild()` - Called on the parent element, requires reference to child

Removal Examples

```
const div = document.querySelector("#root");

// Method 1: Direct removal
div.children[2].remove();

// Method 2: Parent removes child
const secondParagraph = document.querySelector("#root :nth-of-type(2)");
div.removeChild(secondParagraph);

// Method 3: Remove by selector
const elementToRemove = div.querySelector("p:last-child");
elementToRemove.remove();
```

6. Best Practices and Tips

Performance Considerations

- Use `children` instead of `childNodes` when you only need element nodes
- Cache DOM queries in variables to avoid repeated selections
- Use `DocumentFragment` for multiple element insertions

Error Handling

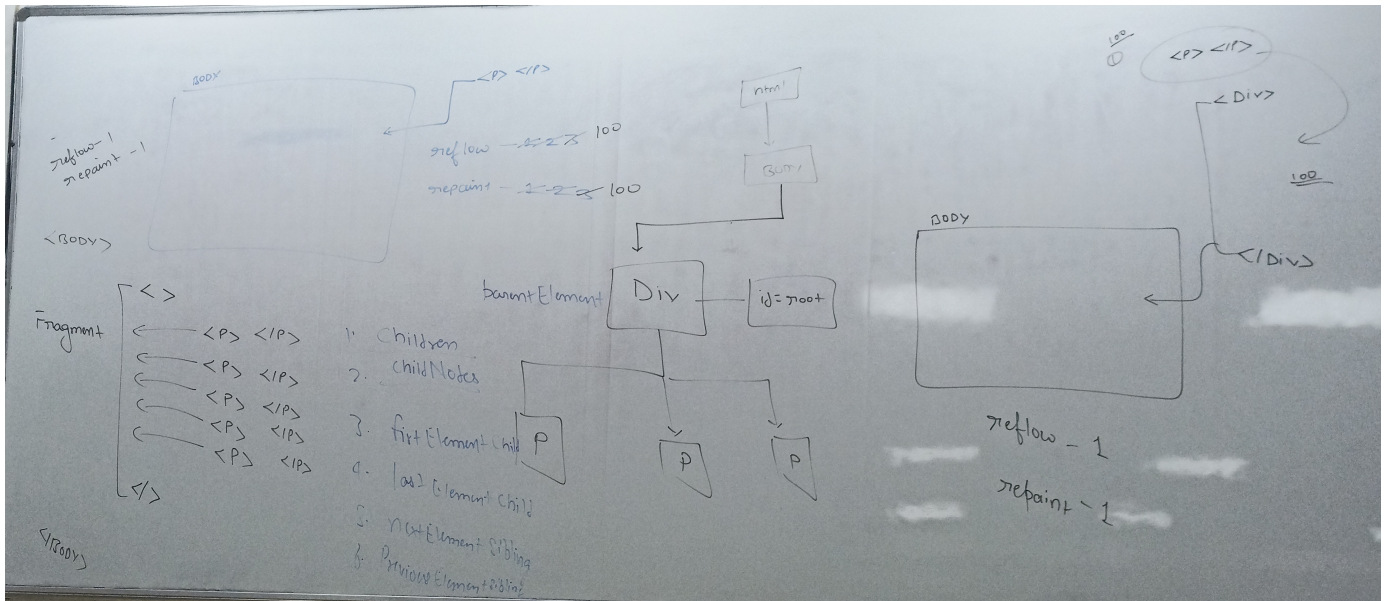
```
// Always check if elements exist before manipulation
const element = document.querySelector("#myElement");
if (element) {
  element.setAttribute("class", "new-class");
}
```

Common Patterns

```
// Creating and inserting multiple elements
const fragment = document.createDocumentFragment();
for (let i = 0; i < 5; i++) {
  const p = document.createElement("p");
  p.textContent = `Paragraph ${i + 1}`;
  fragment.appendChild(p);
}
document.getElementById("root").appendChild(fragment);
```

7. Summary

Method/Property	Purpose	Scope
<code>insertAdjacentElement()</code>	Insert elements at specific positions	Relative positioning
<code>setAttribute()</code>	Set attribute values	Element attributes
<code>getAttribute()</code>	Get attribute values	Element attributes
<code>removeAttribute()</code>	Remove attributes	Element attributes
<code>innerHTML</code>	HTML content inside element	Inner content
<code>outerHTML</code>	Replace entire element	Entire element
<code>innerText</code>	Text content only	Inner text
<code>remove()</code>	Remove element	Self removal
<code>removeChild()</code>	Remove child element	Parent-child removal
Traversal properties	Navigate DOM tree	Element relationships



```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head> ...
7  </head>
8  <body>
9      <div id="root">
10         <p>Paragraph Tag-2</p>
11         <script src="script.js"></script>
12     </div>
13 </body>
14 </html>

```

Handwritten annotations in blue ink:

- Arrows pointing from the `<div id="root">` tag to the `<p>` tag and the `<script>` tag.
- Arrows pointing from the `<p>` tag to the `<script>` tag.
- Arrows pointing from the `<script>` tag to the `</div>` tag.
- Arrows pointing from the `</div>` tag to the `</body>` tag.
- Arrows pointing from the `</body>` tag to the `</html>` tag.

DOM Manipulation - Code Based Notes

1. insertAdjacentElement(position, element)

The `insertAdjacentElement()` method inserts a given element node at a specified position relative to the element it is invoked upon.

Syntax

```
element.insertAdjacentElement(position, elementToInsert)
```

Positions

- `beforebegin` - Before the target element itself
- `afterbegin` - Just inside the target element, before its first child
- `beforeend` - Just inside the target element, after its last child
- `afterend` - After the target element itself

Visual Representation

```
<!-- beforebegin -->  
<div id="target">  
  <!-- afterbegin -->  
  <p>existing content</p>  
  <!-- beforeend -->  
</div>  
<!-- afterend -->
```

Detailed Code Example

```

//! insertAdjacentElement(position, element)

const div = document.getElementById("root");

// Create paragraph elements
const p1 = document.createElement("p");
p1.textContent = "Paragraph - 1";

const p3 = document.createElement("p");
p3.textContent = "Paragraph - 3";

// Insert elements inside the div
div.insertAdjacentElement("afterbegin", p1); // Adds as first child
div.insertAdjacentElement("beforeend", p3); // Adds as last child

// Create heading elements for outside positioning
const t1 = document.createElement("h2");
const t2 = document.createElement("h2");

// Set content and styles
t1.textContent = "Start";
t2.textContent = "End";
t1.style.color = "green";
t2.style.color = "red";

// Insert elements outside the div
div.insertAdjacentElement("beforebegin", t1); // Adds before the div
div.insertAdjacentElement("afterend", t2); // Adds after the div

/*
Result structure:
<h2 style="color: green">Start</h2>      <!-- beforebegin -->
<div id="root">
  <p>Paragraph - 1</p>                  <!-- afterbegin -->
  <p>existing content</p>
  <p>Paragraph - 3</p>                  <!-- beforeend -->
</div>
<h2 style="color: red">End</h2>          <!-- afterend -->
*/

```

2. Attribute Methods

setAttribute(attributeName, attributeValue)

Sets the value of an attribute on the specified element.

getAttribute(attributeName)

Returns the value of a specified attribute on the element.

removeAttribute(attributeName)

Removes the attribute with the specified name from the element.

Detailed Code Example

```

    ///! setAttribute();
    ///! getAttribute();
    ///! removeAttribute();

    const div = document.querySelector("div");

    // Get the current class attribute
    const x = div.getAttribute("class");
    console.log("Current class:", x); // Shows existing classes

    // Add new class while keeping existing ones
    div.setAttribute("class", `${x} justify-center`);

    // Remove the id attribute completely
    div.removeAttribute("id");

    // Check the updated class attribute
    const updatedClass = div.getAttribute("class");
    console.log("Updated class:", updatedClass);

    /*
    Key Points:
    - getAttribute() returns the attribute value or null if not found
    - setAttribute() replaces the entire attribute value
    - To preserve existing values, concatenate with template literals
    - removeAttribute() completely removes the attribute
    */

```

Important Notes:

- If the attribute already exists, `setAttribute()` replaces the previous value
- To preserve existing values, first get the current value and concatenate
- `getAttribute()` returns `null` if the attribute doesn't exist.

3. Content Manipulation Properties

innerHTML

Gets or sets the HTML markup contained within the element.

outerHTML

Gets or sets the HTML markup of the element including the element itself.

innerText

Gets or sets the text content of the element and its descendants.

Detailed Code Example

```

/*
- innerHTML
- outerHTML
- innerText
- textContent
*/

const div = document.querySelector("#root");

// innerHTML - adds HTML content inside the element
div.innerHTML = "<h1>Hello world</h1>"; // Replaces all content inside div

/*
What happens:
Before: <div id="root"><p>old content</p></div>
After:  <div id="root"><h1>Hello world</h1></div>
*/

// outerHTML - replaces the entire element
// div.outerHTML = "<section><h1>New Section</h1></section>";

/*
What would happen with outerHTML:
Before: <div id="root">content</div>
After:  <section><h1>New Section</h1></section>
*/

// innerText - sets only plain text (no HTML)
// div.innerText = "Just plain text";

/*
Differences:
- innerHTML: Parses HTML tags, adds inside element
- outerHTML: Replaces entire element structure
- innerText: Only plain text, ignores HTML tags
- textContent: Similar to innerText but includes hidden elements
*/

```

4. DOM Traversal

Detailed Code Example

```
// traversing an element

const div = document.querySelector("#root");

// Parent element
console.log(div.parentElement);
// Output: Shows the parent element (usually <body>)

// Current element
console.log(div);
// Output: Shows the div element itself

// Children collection (only elements, no text nodes)
console.log(div.children); // HTMLCollection with 3 elements
// Output: HTMLCollection(3) [p, p, p]

// All child nodes (includes text nodes like whitespace/line breaks)
console.log(div.childNodes); // NodeList with 7 nodes (includes empty spaces)
// Output: NodeList(7) [text, p, text, p, text, p, text]

// First and last element children
console.log(div.firstElementChild);
// Output: <p>Paragraph Tag-1</p>

console.log(div.lastElementChild);
// Output: <p>Paragraph Tag-3</p>

// Sibling navigation
console.log(div.firstElementChild.nextElementSibling); // Second paragraph (p2)
// Output: <p>Paragraph Tag-2</p>

console.log(div.lastElementChild.previousElementSibling); // Second paragraph (p2)
// Output: <p>Paragraph Tag-2</p>

/*
Key Differences:
- children: Only element nodes (HTMLCollection) - Count: 3
- childNodes: All nodes including text/whitespace (NodeList) - Count: 7

Why childNodes has 7 items:
1. text node (whitespace before first <p>)
2. <p>Paragraph Tag-1</p>
3. text node (whitespace between p tags)
4. <p>Paragraph Tag-2</p>
5. text node (whitespace between p tags)
6. <p>Paragraph Tag-3</p>
7. text node (whitespace after last <p>) */
```

5. Element Removal

remove()

Removes the element from the DOM (called on the element itself).

removeChild(childElement)

Removes a specified child element from the parent (called on parent element).

Detailed Code Example

```

/**
 * ! remove()
 * ! removeChild()
 */

const div = document.querySelector("#root");

// Method 1: Direct removal using remove()
// div.children[2].remove(); // This would remove the third child element

/*
How remove() works:
- Called directly on the element you want to remove
- Element removes itself from its parent
- Simple and direct approach
*/

// Method 2: Parent removes child using removeChild()
const p2 = document.querySelector("#root :nth-of-type(2)"); // Selects 2nd paragraph
div.removeChild(p2); // Parent div removes the p2 element

/*
How removeChild() works:
- Called on the parent element
- Requires reference to the child element
- Parent explicitly removes the specified child
- Returns the removed element (can be stored in variable)

Selector explanation:
"#root :nth-of-type(2)" means:
- Inside element with id="root"
- Find the 2nd element of its type (2nd <p> tag)
*/

// Alternative removal methods:
// Remove by index
// div.children[1].remove();

// Remove first child
// div.firstElementChild.remove();

// Remove last child
// div.lastElementChild.remove();

/*
Key Differences:
remove():
- element.remove()
- Called on element itself
- Modern method (ES6+)
- Simpler syntax

removeChild():
- parent.removeChild(child)
- Called on parent element
- Older method (works in all browsers)
- Returns removed element
- Need reference to both parent and child
*/

```

6. Best Practices and Tips

Performance Considerations

- Use `children` instead of `childNodes` when you only need element nodes
- Cache DOM queries in variables to avoid repeated selections
- Use `DocumentFragment` for multiple element insertions

Error Handling

```
// Always check if elements exist before manipulation
const element = document.querySelector("#myElement");
if (element) {
  element.setAttribute("class", "new-class");
}
```

Common Patterns

```
// Creating and inserting multiple elements
const fragment = document.createDocumentFragment();
for (let i = 0; i < 5; i++) {
  const p = document.createElement("p");
  p.textContent = `Paragraph ${i + 1}`;
  fragment.appendChild(p);
}
document.getElementById("root").appendChild(fragment);
```